

LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior Software Engineer with 9+ years of experience delivering production backend services with Go and complementary Node.js/TypeScript systems, plus pragmatic React UIs when needed. Strong focus on clean API contracts, observability-first delivery, and performance work using caching, indexing, and efficient data-access patterns. Experienced in Microsoft Azure deployments, CI/CD automation, and incident-ready operations that reduce time-to-diagnosis and improve release confidence. Known for making careful trade-offs, writing maintainable service boundaries, and collaborating with product and platform stakeholders to ship reliable capabilities without over-engineering.

SKILLS

- Go (Golang)
- Go Concurrency (Goroutines, Channels)
- Context Propagation & Cancellation
- Go Profiling (pprof) & Performance Tuning
- RESTful WebServices
- API Contract Design
- Versioned APIs & Backward Compatibility
- Pagination, Filtering & Sorting Standards
- Idempotency Keys & Safe Retries
- Structured Error Modeling
- Authentication & Authorization (OAuth2, JWT)
- Role-Based Access Control (RBAC)
- Rate Limiting & Throttling
- Background Processing & Job Queues
- Event-Driven Patterns
- Webhook Design & Signature Verification
- gRPC Fundamentals
- Message Serialization (JSON)
- Integration Patterns
- Dependency Injection Patterns
- Clean Architecture & Layered Services
- Distributed Systems Debugging
- Microsoft Azure
- Azure App Service Concepts
- Azure Storage Concepts
- Azure Networking Concepts
- Managed Identity Concepts
- Secrets Management Concepts
- Containerized Deployments (Docker)
- Environment Separation (Dev/Staging/Prod)
- Blue-Green Deployment Strategies
- Release Rollouts & Feature Flags
- Cloud Monitoring & Alerting
- Cost-Aware Scaling Considerations
- Resilience Patterns (Timeouts, Circuit Breakers)
- SQL
- PostgreSQL
- Schema Design
- Indexing Strategies
- Query Tuning
- Transactions & Consistency
- Connection Pooling Concepts
- Database Migration Practices
- Redis
- Caching Strategies
- Cache Invalidation Patterns
- Read-Through & Write-Through Cache Patterns
- CI/CD Pipelines
- GitHub Actions
- Automated Build & Release Workflows
- Container Build Optimization
- Operational Dashboards
- Runbooks & Incident Playbooks
- Metrics-Driven Reliability Improvements
- Infrastructure-as-Code Concepts
- Release Coordination & Rollback Readiness
- Structured Logging & Correlation IDs
- Traceability Across Service Boundaries
- Golden Signals Monitoring
- Alert Tuning to Reduce Noise
- Postmortems & Corrective Action Tracking
- React
- TypeScript
- JavaScript (ES6+)
- Component-Driven UI Architecture
- Reusable UI Libraries & Design Systems
- State Management Patterns
- Accessible UI Implementation (WCAG)
- Performance Profiling & Render Optimization
- Complex Forms & Validation Patterns
- Responsive Layout Engineering
- HTML5
- CSS3
- Test Strategy Design (Unit, Integration, Contract)
- Integration Testing
- End-to-End Testing Practices
- Mocking & Test Doubles
- Regression Prevention Workflows
- Test Data Management Practices

WORK HISTORY

Senior Software Engineer
RavenTrack

August 2021 to November 2025

- Led design and delivery of Go services that handled core product workflows, shaping stable interfaces and ensuring downstream teams could integrate without brittle assumptions or hidden coupling.

- Implemented consistent REST contracts in Go, including pagination, filtering, and deterministic sorting rules, so clients could rely on predictable behavior across releases and environments.
- Built robust authentication flows with JWT and well-scoped authorization checks, aligning service permissions with real product roles while preventing privilege escalation through edge-case requests.
- Improved performance of high-traffic endpoints by combining PostgreSQL query tuning with Redis caching, then validated gains using repeatable load tests and production telemetry comparisons.
- Introduced correlation IDs and structured logging conventions across services, making cross-request tracing reliable during incidents and reducing time spent guessing where failures originated.
- Designed resilient integration patterns for third-party APIs, adding idempotency keys, retry policies, and backoff behavior so transient failures did not create duplicate records or noisy outages.
- Hardened service reliability by standardizing timeouts and cancellation using Go contexts, preventing resource leaks under load and ensuring downstream dependencies could be safely abandoned.
- Delivered Azure-first deployments with environment separation and safe rollout patterns, enabling rapid releases while keeping rollback paths practical when an unexpected production issue occurred.
- Created CI/CD workflows that enforced formatting, linting, tests, and artifact promotion rules, reducing manual release steps and eliminating several recurring deployment-related regressions.
- Collaborated with product and operations stakeholders to turn ambiguous requirements into deliverable slices, documenting trade-offs so future engineers understood why decisions were made.
- Refactored legacy modules into clearer boundaries with explicit interfaces, reducing accidental complexity and making it easier to test critical business rules without heavy integration setup.
- Established API error standards that mapped internal failures into stable client responses, ensuring consumers received actionable error codes rather than inconsistent messages or stack traces.
- Built background processing pipelines for deferred tasks, ensuring long-running operations were observable and recoverable while keeping user-facing requests fast and predictable under spikes.
- Developed operational dashboards and alert thresholds based on service-level signals, then iterated on alert noise so on-call focus stayed on actionable incidents rather than symptoms.
- Mentored engineers through design reviews and production debugging, sharing practical patterns for Go services and helping the team ship changes that stayed maintainable after feature pressure.

Software Engineer Infinite Loop

June 2018 to July 2021

- Delivered Go-based backend capabilities that supported evolving product requirements, focusing on explicit contracts and data validation so client teams could build features confidently.
- Implemented REST endpoints with careful request parsing and consistent response shapes, ensuring compatibility across versions while limiting breaking changes through deliberate deprecation steps.
- Built integration services that coordinated multiple upstream systems, handling partial failures with retries and compensating logic so business workflows stayed consistent during outages.
- Optimized PostgreSQL-backed operations by introducing better indexes and query patterns, then verified improvements using measured timings instead of relying on anecdotal performance reports.
- Added Redis caching where it provided clear value, establishing cache keys and invalidation rules that balanced freshness with speed and avoided silent correctness regressions.
- Improved reliability by strengthening error handling and making failures observable, ensuring logs carried enough context to reproduce issues without requiring direct database access.
- Collaborated with Azure-hosted delivery pipelines, refining build and release steps so deployments were repeatable and environment drift became visible rather than discovered late.
- Implemented secure authentication and authorization checks aligned with role permissions, preventing unintended data exposure through overly broad endpoints or weak filtering rules.
- Worked closely with front-end stakeholders to align API payloads to real UI needs, reducing over-fetching and repeated calls that previously caused latency and inconsistent state.
- Introduced practical testing coverage with unit and integration focus, catching regressions in request validation and database behavior before changes reached production traffic.
- Participated in incident response and post-incident improvements, translating production learnings into concrete fixes that reduced recurrence rather than producing purely procedural documents.

Software Developer Datamart

October 2015 to May 2018

- Built and maintained backend features with a focus on reliable CRUD workflows, ensuring data entry and reporting tools behaved consistently for daily users handling large datasets.
- Implemented SQL-backed reporting with careful schema and index choices, reducing slow queries that impacted peak business hours and improving overall responsiveness for analysts.
- Developed service endpoints and internal APIs with clear request validation, preventing malformed data from entering core tables and reducing downstream cleanup and reconciliation work.
- Improved stability by standardizing error handling and logging practices, making recurring failures easier to diagnose without repeatedly escalating to database administrators.
- Collaborated with stakeholders to translate business rules into implementable logic, capturing edge cases early to reduce late rework and avoid user-facing behavior surprises.

- Introduced incremental performance improvements through query tuning and batching, ensuring large exports and reconciliations completed within operational windows without timing out.
- Contributed to deployment and release hygiene by documenting runbooks and rollback steps, helping teams execute changes safely even when senior staff were unavailable.
- Supported integration points between internal systems, building dependable data flows and sanity checks so upstream changes did not silently corrupt reports or dashboards.
- Raised overall code maintainability by refactoring high-churn modules, creating clearer boundaries that simplified future updates and reduced the risk of unintended side effects.

EDUCATION

Bachelor's degree in Computer Science
University of Amsterdam

2011 to 2015